

```
// === Memory Match (js/games/memory.js) ===
```

```
/**
 * Memory Match (<°_Æ•ûrL) - 5 Qs•iQsj!_
 * @author ch2077
 * @version 2.0.0
 * @date 2025-04-08 R Qe•iQsy
 * @da
```

```
    this.seconds++;
```

```
    this.updateStats();
```

```
    }, 1000);
```

```
    },
```

```
    updateStats() {
```

```
        if (this.statsCb) {
```

```
            const cfg = this.levelCon
```

```
/**
 * Whack-a-Mole (bSw0Y ) - 5 Qs•iQsj!_
 * @author ch2077
 * @version 2.0.2
 * @date 2025-04-22 R Qe•iQs|û~ßY vîh R ep+eö•ô-PR6
 * @date 2025-05-01 0 S SIW0Y i,s†-•QsSa• Xž
 *
 * 30y0-PeöbSw0Y y 9N*m Să-•g:Q
 * kĬN Qsg vîh R epY •%h
 * Qúm • ^!|-•QsSaR • y
 */
const
```

```
if (this.moles[idx]) {
```

```
    this.moles[idx] = false;
```

```
    this.score += 10;
```

```
    const mole = document.getElementByIdBy
```

```
this.running = false;
```

```
clearInterval(this.timer);
```

```
clearTimeout(this.spawnTimer);
```

```
if (this.score > this.bestScore) {
```

```
th
```

␣

```
// Bind keyboard clicks␣
```

```
this.container.querySelectorAll('.wordle-key').forEach(btn => {␣
```

```
btn.addEventListener('click', () =>
```

```
if (result[i] === 'correct') continue;ð
```

```
const idx = targetArr.indexOf(guess[i]);ð
```

```
if (idx !== -1) {ð
```

```
result[i] = 'p
```

```
  }},
  bp: { name: '€İS ', icon: 'Ø<ß'', items: [
    '•{W<€İS ', 'N-W<€İS ', 'Y'W<€İS ', 'b g/€İS ', 'z•QÜ€İS ', ' 'îb €İS ', ' ^LQ>€İS '
  ]},
  chest: { name: '€øc ', icon: 'Ø>Ý°', items: [
    '•{W<€øc ', 'N
```



```
document.getElementById('rollBtn').addEventListener('click', () => this.startR
    },
    startRoll() {
        if (this.spinning) return;
```

```
* @date 2025-02-10 e9u(Vpn~lõu b [Æeteprî+c zz
    * @date 2025-03-05 OiN†-¼^|• béeögenerateSak{v,,•i~~
        *
            * 9x9h QÆepriÿ 3yi-¼^!ÿ easy/medium/hardÿ
                * u b {-lõÿ Vpn~lõ• _Ru b [Æet%ãÿ Q•c -¼^|c c%,â^rh<
                    * ŠÆ<Áÿ ^L/R /[«T 1-9N 'íY
                        */
                            const SudokuGame = {
                                board: [],
                                solution: [],
```

```
render() {  
  this.container.innerHTML = `  
    <div style="text-align:center">  
      <div class="sudoku-board" style="display:grid
```

```
    },  
    refreshAllCells() {  
        const cells = this.container.querySelectorAll('.sudoku-board div');  
        cells.forEach(cell => {  
            const
```

```
        container: null,
        statsCb: null,

        init(container, statsCb) {
            this.container = container;
            this.statsCb = statsCb;
            this.score
```

```
    },  
    draw() {  
        const c = this.ctx;  
        c.clearRect(0, 0, this.width, this.height);  
  
        // Sky  
        c.fillStyle = '#1a3a5c';  
        c.fillRect(0, 0, this.width, 100);  
    }  
}
```

```
* @date 2025-03-22 R N†p,_9`é•Z+u T}|û~ß
*
* nŃR`{î[Pc¥00C%,=l4gøý ••_ p,_9
* l4gø8yíemoji-•g:ý p,_9i,s†15%ý • ^!-•eö•ö• Xž
* 5gaT}ý o c¥l4gø-1T}ý c¥R0p,_9vôc¥-1T}
*/
const FruitCatchGame = {
    canvas: null,
    ctx: null,
    width: 360,
    height: 550,
    basket
```

```
    this.loop = setInterval(() => this.tick(), 16);  
  },
```

```
  clampBasket() {
```

```
    this.basket.x = Math.max(0, Math.min(this.width - this.bas
```



```
if (this.statsCb) {
```

```
    this.statsCb({ '_R ': this.score, 'u T': this.lives });
    }
    },
```

```
gameOver() {
    this.running = false;
```

```
clear
```

```
render() {  
    this.container.innerHTML = '<div class="puzzle-grid"></div>';  
    this.gridEl = this.container.querySelector('.puzzle-gr
```

```
player: 'X',  
  ai: 'O',  
  turn: 'X',  
  wins: 0,  
  losses: 0,  
  draws: 0,  
  container: null,  
  statsCb: null,  
  gridEl: null,  
  init(container)
```

```
// Minimax
```

```
    let best = -Infinity;
```

```
    let move = 0;
```

```
    for (let i = 0; i < 9; i++) {
```

```
        if (this.board[i] === '') {
```

```
            this.
```

showingSequence: false,0

score: 0, highScore: 0,0

container: null, statsCb: null,0

colors: ['#ff6b6b', '#54a0ff', '#00d2a0', '#ffd93d'],

␣

```
if (idx !== this.sequence[this.playerIdx]) {␣
```

```
document.getElementById('simonStatus').innerHTML = '<span style="color:#ff6b
```

```
zone.addEventListener('click', () => {  
    if (this.state === 'idle' || this.state === 'clicked') {  
        this.startCountdown();
```

```
updateStats() {  
    if (this.statsCb) {  
        this.statsCb({ 'g Os': this.best === Infinity ? '--' : `${this.best}ms` });  
    },  
    destr
```



```
badge.style.background = 'var(--green)';
```

```
badge.style.color = '#fff';
```

```
badge.textContent = val + ' ' ' ';
```

```
history.append
```

```
    this.running = true;
```

```
    this.render();
```

```
    this.nextRound();
```

```
    this.startTimer();
```

```
    this.updateStats();
```

```
  },
```

```
  render() {
```

```
    this.c
```

```
    }  
    },  
  
    gameOver() {  
        this.running = false;  
        clearInterval(this.timer);  
        if (this.streak > this.bestStreak) this.bestStreak =
```

```
.app-header h1 { font-size: 20px; font-weight: 700; color: var(--text); }  
.  
.btn-back {  
    background: var(--surface2);  
    color: var(--text);
```

␣

.btn {␣

background: var(--accent);␣

color: #fff;␣

border: none;␣

padding: 8px 18px;␣

border-radius: 8px;␣

cursor: pointer;␣

font-size:

font-weight: 700;␣

transition: all 0.1s ease;␣

}␣

␣

/* === Minesweeper === */␣

.mine-grid {␣

display: grid;␣

gap: 2px;␣

background: var(--sun

```
gap: 16px;
}
.game-over-overlay h3 { font-size: 24px; }
/* === Level Transition Overlay === */
.level-intro {
position: absolute;
inset: 0 0 0 0;
```

```
.delta-slot {  
  background: var(--surface2);  
  border: 2px solid var(--border);  
  border-radius: var(--radius);  
  padding: 12px;  
  text-align: center;  
}
```



```
align-items: center;␣
    gap: 8px;␣
        padding: 4px 0;␣
            border-bottom: 1px solid var(--border);␣
        }␣
        .delta-result-item:last-child { border-bottom: 1px solid var(--border); }
```

```
␣
/* === D-Pad (Snake) === */␣
    .dpad {␣
        display: grid;␣
            grid-template-areas: ". up ." "left . right" ". down .";␣
                grid-template-columns: repeat(3, 1fr);
```